EXPERIMENT 2

Shravanya Andhale

D20A-02

Text Preprocessing, Tokenization and Word Embeddings for Natural Language Processing

AIM:

To perform text preprocessing, tokenization, and word embedding techniques for Natural Language Processing (NLP) tasks using Python libraries, and analyze their importance in preparing textual data for Large Language Models (LLMs) and transformer-based applications.

THEORY:

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, process, analyze, and generate human language. Human language is complex and unstructured, containing variations in spelling, punctuation, grammar, word forms, and sentence structures. Therefore, before textual data can be used by machine learning models or Large Language Models, it must be converted into a suitable numerical representation.

The major steps involved in preparing textual data include text preprocessing, tokenization, and word embedding. These techniques help transform raw text into a structured format that can be processed by computational models.

Text Preprocessing

Text preprocessing is the process of cleaning and transforming raw textual data before it is used for Natural Language Processing tasks. Real-world text may contain unnecessary symbols, punctuation, special characters, inconsistent capitalization, stop words, and other forms of noise.

Common text preprocessing techniques include:

- Converting text to lowercase
- Removing punctuation
- Removing special characters
- Removing unnecessary spaces
- Removing stop words
- Stemming
- Lemmatization

For example, the sentence:

> “The Students are Learning NLP!!!”

may be converted into a simplified form such as:

> “student learn nlp”

depending on the preprocessing techniques applied.

The purpose of preprocessing is to reduce noise and create more consistent input data. However, modern transformer-based models sometimes require minimal preprocessing because their pretrained tokenizers are designed to handle punctuation, capitalization, and contextual information.

Tokenization

Tokenization is the process of dividing text into smaller units called **tokens**. A token may represent a word, subword, character, or symbol.

For example:

> "Natural Language Processing is interesting."

can be divided into word tokens as:

```
["Natural", "Language", "Processing", "is", "interesting"]
```

Tokenization is an essential step because machine learning models cannot directly process raw sentences. The input text must first be converted into smaller units that can later be represented numerically.

Different types of tokenization include:

- **Word tokenization** – divides text into individual words.
- **Sentence tokenization** – divides a paragraph into individual sentences.
- **Character tokenization** – divides text into individual characters.
- **Subword tokenization** – divides words into smaller meaningful units.

Modern transformer models such as BERT and GPT commonly use subword tokenization methods. Subword tokenization helps models handle rare words and previously unseen words more effectively.

Stop Words

Stop words are commonly occurring words that may provide limited information for certain NLP tasks. Examples include:

> "the", "is", "and", "in", "of"

Removing stop words can reduce the size of the text data and may be useful in traditional NLP applications such as document classification and information retrieval.

However, stop-word removal is not always appropriate. In transformer-based models, even common words can contribute important contextual information. Therefore, preprocessing techniques should be selected according to the requirements of the NLP task and model.

Stemming

Stemming is a text normalization technique that reduces words to their root or base form by removing prefixes or suffixes.

For example:

- Playing → Play
- Played → Play
- Playing → Play

Stemming is generally based on rules and may sometimes produce incomplete or linguistically incorrect words. Despite this limitation, it can be useful for reducing variations of similar words.

Lemmatization

Lemmatization is another technique used to convert words into their meaningful base form, known as a **lemma**.

For example:

- Running → Run
- Better → Good
- Studies → Study

Unlike simple stemming, lemmatization considers the linguistic meaning and grammatical form of a word. Therefore, it generally produces more meaningful results.

Word Embeddings

Computers cannot directly understand the meaning of words. Therefore, words and tokens must be converted into numerical representations before they can be processed by machine learning models.

A word embedding is a numerical vector representation of a word. Unlike simple numerical encoding methods, embeddings attempt to capture semantic and contextual relationships between words.

For example, words with similar meanings may have vector representations that are closer together in an embedding space.

Popular word embedding techniques include:

- **Word2Vec**
- **GloVe**
- **FastText**

Word2Vec, available through libraries such as Gensim, learns vector representations of words based on their surrounding context. It includes two common architectures:

1. **Continuous Bag of Words (CBOW)** – predicts a target word using surrounding words.
2. **Skip-gram** – predicts surrounding words based on a target word.

Importance of Word Embeddings

Word embeddings provide several advantages over traditional text representations such as one-hot encoding.

In one-hot encoding, every word is represented by a large vector containing mostly zeros. These representations do not capture semantic relationships between words.

Word embeddings provide dense numerical vectors that can capture relationships between words. For example, words such as “king” and “queen” or “cat” and “dog” may have meaningful relationships within the embedding space.

Word Embeddings and Transformer Models

Traditional embedding techniques such as Word2Vec generate a fixed representation for each word. This means that a word generally has the same vector regardless of its context.

Transformer-based models, however, generate contextual embeddings. The representation of a word can change depending on the surrounding words.

For example, the word “bank” has different meanings in:

- “I deposited money in the bank.”
- “He sat near the river bank.”

Transformer models can use contextual information to generate different representations for the same word depending on its meaning in the sentence.

Importance in Large Language Models

Text preprocessing, tokenization, and embeddings form an important foundation for Natural Language Processing and Large Language Models. Before a transformer model processes text, the input is typically converted into tokens The general process can be represented as:

Raw Text → Preprocessing → Tokenization → Numerical Representation/Embeddings → Transformer Model → Output

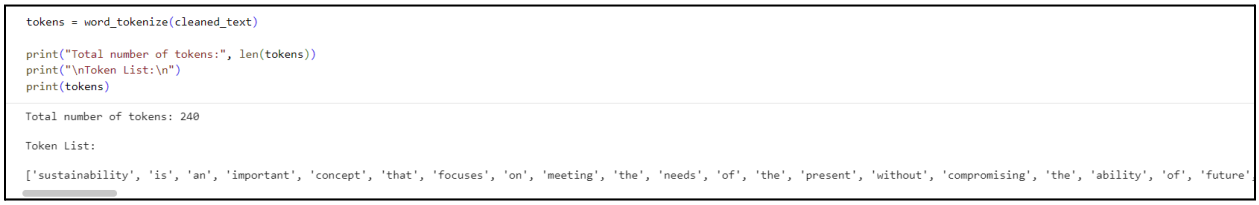
The quality and appropriateness of these steps can influence the performance of an NLP system. Traditional NLP models may require extensive preprocessing, whereas pretrained transformer models usually rely on their own specialized tokenizers and embedding mechanisms.

Parameter	Range / Values	Interpretation
Text Case	Uppercase / Lowercase / Mixed case	Lowercasing was used to ensure that words with different capitalization are treated consistently.
Punctuation Handling	Retained / Removed	Punctuation was removed to reduce unnecessary symbols and obtain cleaner text.

Tokenization Type	Sentence / Word / Character / Subword	Word tokenization was used to divide the text into individual words for further processing.
Stop-word Removal	Applied / Not Applied	Stop words were removed to retain more meaningful and informative words for analysis.
Stemming Method	Porter / Lancaster / Snowball	Porter Stemmer was used to reduce words to root-like forms.
Lemmatization	Applied / Not Applied	WordNet Lemmatizer was used to convert words into meaningful base forms.
Word Frequency	1 to total number of occurrences	A higher frequency indicates that a word occurs more often in the processed text and may represent an important concept.
Embedding Method	Word2Vec / GloVe / FastText	Word2Vec was used to convert words into dense numerical vector representations based on contextual relationships.
Word Similarity	−1 to +1 (approximately, cosine similarity)	Higher positive values indicate greater similarity between word vectors, while lower values indicate weaker relationships.
Embedding Context	Small to Large training corpus	A larger and more diverse corpus generally produces more meaningful word relationships. In this experiment, the training text was relatively small, so similarity results may be limited.

CODE AND OUTPUT:

https://colab.research.google.com/drive/10PEsY36m-F3lhq8IUgcEZqtXqKGn5ApC?usp=sharing



```
tokens = word_tokenize(cleaned_text)

print("Total number of tokens:", len(tokens))
print("\nToken List:\n")
print(tokens)
```

```
Total number of tokens: 240

Token List:

['sustainability', 'is', 'an', 'important', 'concept', 'that', 'focuses', 'on', 'meeting', 'the', 'needs', 'of', 'the', 'present', 'without', 'compromising', 'the', 'ability', 'of', 'future',
```

```
stop_words = set(stopwords.words('english'))

processed_tokens = [
    word for word in tokens
    if word.isalpha() and word not in stop_words
]

print("Number of tokens after stop-word removal:", len(processed_tokens))
print("\nProcessed Tokens:\n")
print(processed_tokens)
```

```
Number of tokens after stop-word removal: 169

Processed Tokens:

['sustainability', 'important', 'concept', 'focuses', 'meeting', 'needs', 'present', 'without', 'compromising', 'ability', 'future', 'generations', 'meet', 'needs', 'sustainable', 'society', 'uses
```

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [
    lemmatizer.lemmatize(word)
    for word in processed_tokens
]

print("Lemmatized Words:\n")
print(lemmatized_words)
```

```
Lemmatized Words:

['sustainability', 'important', 'concept', 'focus', 'meeting', 'need', 'present', 'without', 'compromising', 'ability', 'future', 'generation', 'meet', 'need', 'sustainable', 'society', 'us',
```

```
ORIGINAL TOKENS:

['sustainability', 'is', 'an', 'important', 'concept', 'that', 'focuses', 'on', 'meeting', 'the', 'needs', 'of', 'the', 'present', 'without', 'compromising', 'the', 'ability', 'of', 'future',

PREPROCESSED TOKENS:

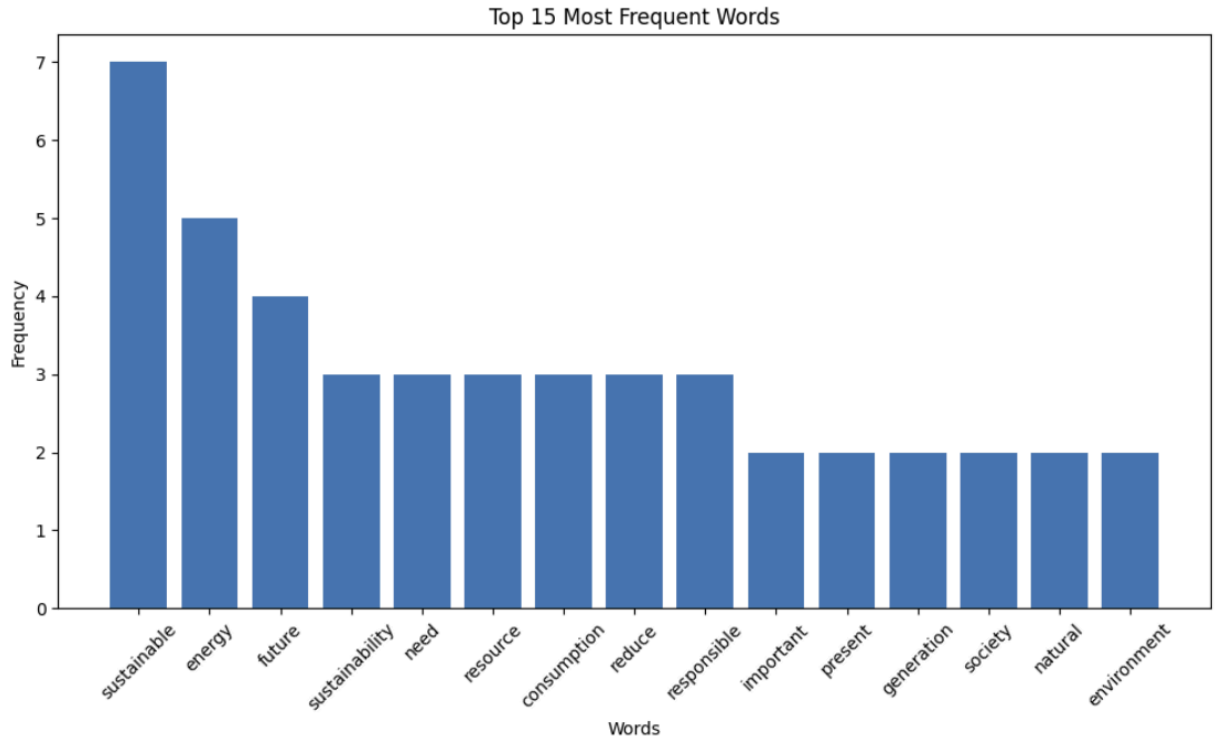
['sustainability', 'important', 'concept', 'focuses', 'meeting', 'needs', 'present', 'without', 'compromising', 'ability', 'future', 'generations', 'meet', 'needs', 'sustainable', 'society',

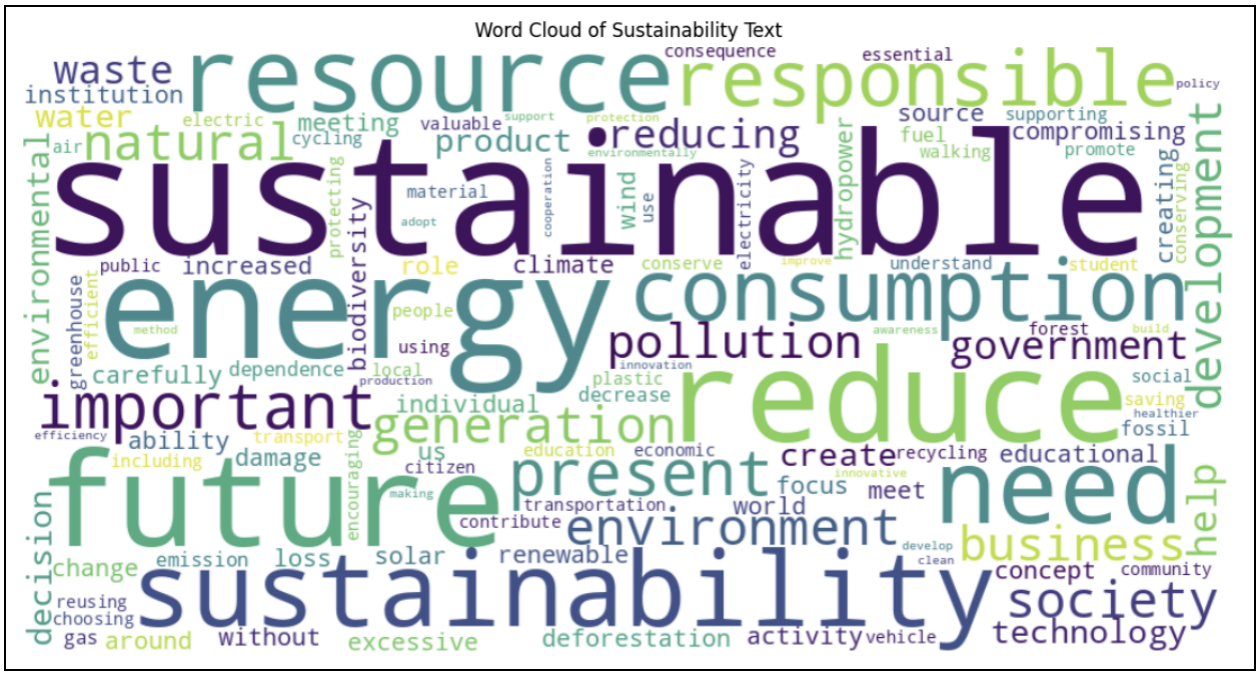
STEMMED WORDS:

['sustain', 'import', 'concept', 'focus', 'meet', 'need', 'present', 'without', 'compromis', 'abil', 'futur', 'gener', 'meet', 'need', 'sustain', 'societi', 'use', 'natur', 'resourc', 'care',

LEMMATIZED WORDS:

['sustainability', 'important', 'concept', 'focus', 'meeting', 'need', 'present', 'without', 'compromising', 'ability', 'future', 'generation', 'meet', 'need', 'sustainable', 'society', 'us',
```






```
Embedding vector for 'sustainability':
[ 0.1539933  -0.09041538 -0.15817718 -0.25520882 -0.0845453  -0.31183025
  0.15592736  0.3350815  -0.10901681 -0.11582921  0.01141961 -0.25547805
  0.22323626  0.3080543  -0.08074539  0.04218852  0.00725146  0.13204111
 -0.41406277 -0.14535978  0.30651766  0.14394818  0.5369163  -0.09673214
  0.25027528 -0.14467312 -0.08792467 -0.05542356 -0.27964124  0.10037877
  0.04092082 -0.09629984 -0.02966314 -0.0822175  -0.0527559  -0.05127519
  0.26250818  0.15954308 -0.03183643  0.06942341  0.1490373  -0.07866287
  0.16328229 -0.19544733  0.41289747  0.0727293  -0.0494407  -0.1115906
  0.0775613  -0.00087813]

Embedding vector for 'environment':
[ 0.13987865 -0.05473896  0.008616   -0.18819894  0.01635117 -0.3150826
  0.16615999  0.40231735 -0.1881952  -0.09198078  0.02157129 -0.24599361
  0.07568721  0.26580775  0.00169989  0.10011095  0.03947229  0.14406887
 -0.3619948  -0.1709517   0.16560718  0.13088998  0.5676148  -0.20083582
  0.17578451 -0.07648616 -0.05573128 -0.08173656 -0.14663959 -0.00374764
 -0.01530558 -0.13737008 -0.00535166 -0.05915349 -0.06089379  0.08489019
  0.24515404  0.10072716 -0.07547367  0.05479556  0.29065773 -0.0601547
  0.04444698  0.04720929  0.28530496  0.04484308 -0.07702159 -0.08528784
  0.07714038  0.09648495]

Embedding vector for 'energy':
[-0.00884553  0.03025808 -0.29719144 -0.17964873  0.3865103  -0.41607538
  0.00173547  0.21382076 -0.35534298  0.0066131  -0.05777695 -0.1680785
  0.26479092  0.21623275  0.19074422  0.09548169 -0.0518049  -0.04089661
 -0.31420153  0.04299695  0.23581217  0.19326212  0.5664508  -0.42398128
  0.3627326  -0.2993555  -0.01759981 -0.273236   -0.38960478 -0.28520498
  0.06730816  0.01550892 -0.3116587   0.05144041 -0.2625889  -0.10617098
  0.16552082  0.13481241 -0.19761309  0.00281437  0.1708083   0.17497186
 -0.07081616 -0.0415284   0.37344208  0.24618891 -0.33002225  0.04843413
```

```
    print(
        f"Similarity between '{word1}' and '{word2}': "
        f"{similarity_score:.4f}"
    )
```

```
Similarity between 'sustainability' and 'environment': 0.9088
```

```
============================================================
NLP TEXT PREPROCESSING AND WORD EMBEDDING RESULTS
============================================================

1. Original word count: 283
2. Processed word count: 169
3. Number of unique processed words: 127
4. Word2Vec vocabulary size: 127

Top 10 Frequent Words:
sustainable: 7
energy: 5
future: 4
sustainability: 3
need: 3
resource: 3
consumption: 3
reduce: 3
responsible: 3
important: 2
```

Preprocessing Results and Discussion

The original paragraph contained complete sentences with punctuation, capitalization, and commonly occurring words such as *the*, *is*, *and*, and *of*. After converting the text to lowercase, similar words with different capitalization were treated consistently. For example, words appearing at the beginning of sentences were no longer considered different from the same words appearing elsewhere in the text.

Punctuation marks and special characters were removed to create a cleaner textual representation. Tokenization then divided the paragraph into individual words. This made it possible to analyze each word separately and apply further preprocessing operations.

Stop-word removal reduced the number of unnecessary words in the dataset. The remaining words mainly represented important concepts related to the topic, such as sustainability, environment, energy, resources, development, water, and future. This process helped focus the analysis on meaningful information.

Stemming was performed using the Porter Stemmer. The stemming process reduced words to shortened root-like forms by removing suffixes. Although stemming reduces the number of different word forms, some resulting words may not appear as proper English words. Lemmatization was also performed using the WordNet Lemmatizer. Compared with stemming, lemmatization produced more meaningful dictionary-based forms and preserved better linguistic structure.

Word frequency analysis identified the most frequently occurring words in the processed text. The frequency results highlighted the main concepts discussed in the sustainability paragraph. A Word Cloud was generated to provide a visual representation of these frequently occurring words. Words with higher frequency appeared more prominently, making it easier to identify the major themes of the text.

Word2Vec was used to generate numerical vector representations of words. These embeddings represent words as dense vectors based on their contextual relationships within the training data. Embedding vectors were generated for selected words such as sustainability, environment, energy, development, and future. The trained model was also used to identify words that were most similar to a selected keyword.

Since the Word2Vec model was trained using a relatively small sustainability paragraph, the similarity results may not be as accurate as those generated by models trained on large datasets. However, the experiment successfully demonstrated how words can be converted into numerical representations and how relationships between words can be analyzed.

CONCLUSION:

Text preprocessing, tokenization, and word embeddings are fundamental techniques in Natural Language Processing. Preprocessing helps clean and normalize textual data, tokenization divides text into meaningful units, and word embeddings convert language into numerical representations that can capture semantic relationships. These techniques provide an important foundation for preparing textual data for machine learning, deep learning, transformer models, and Large Language Models.